

Task 10: Session 10 (AIML) - Assignment Questions

Name: Mihir Sopan Patil

Domain: AIML

College & Branch Name: Zeal Polytechnic, Diploma In AIML

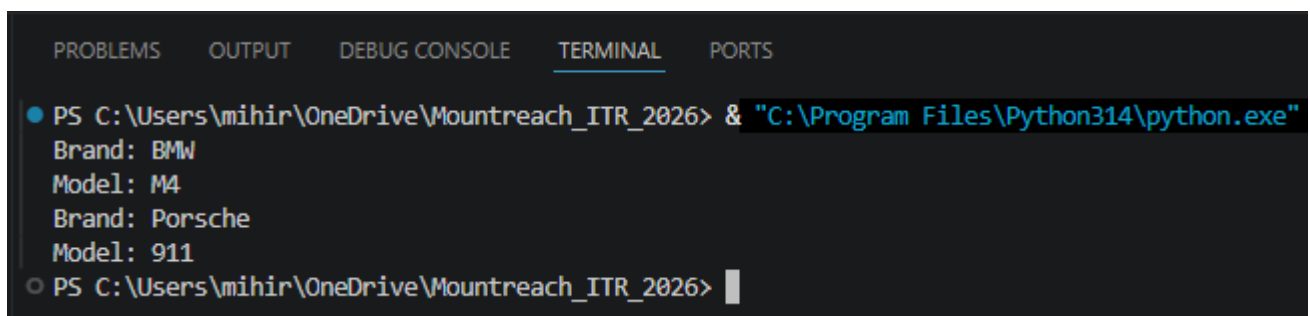
1. Basic Class & Object

Create a class named Car with:

- Attributes: brand and model (set using `__init__`).
- A method `display_info()` that prints the brand and model.

Create two objects of the Car class with different values and call the method for both.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Brand: BMW
Model: M4
Brand: Porsche
Model: 911
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        print("Brand:", self.brand)
        print("Model:", self.model)

car1 = Car("BMW", "M4")
car2 = Car("Porsche", "911")
car1.display_info()
car2.display_info()
```

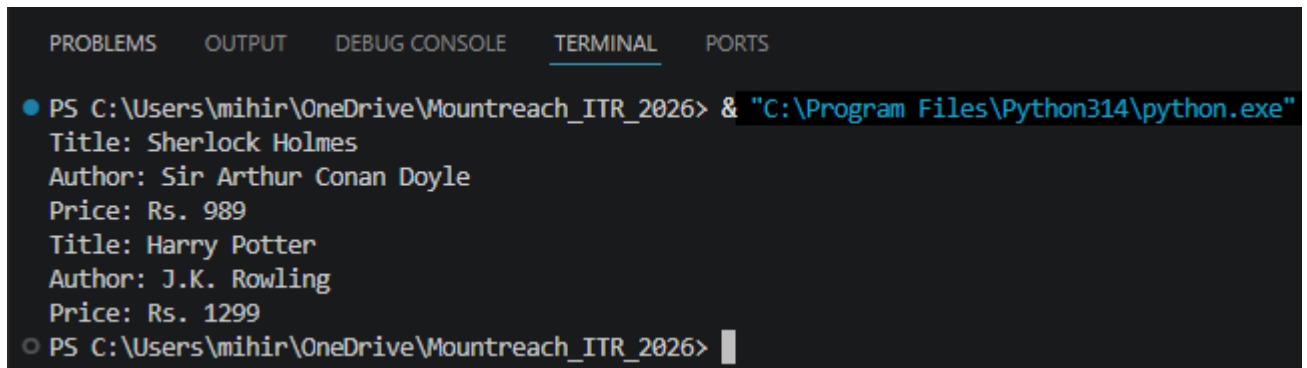
2. Using init Constructor

Write a class Book that takes title, author, and price as parameters in `__init__`.

Create a method `show_details()` to print all the book information.

Create at least two book objects and display their details.

Output:



```
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Title: Sherlock Holmes
Author: Sir Arthur Conan Doyle
Price: Rs. 989
Title: Harry Potter
Author: J.K. Rowling
Price: Rs. 1299
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price

    def show_details(self):
        print("Title:", self.title)
        print("Author:", self.author)
        print("Price: Rs.", self.price)

book1 = Book("Sherlock Holmes", "Sir Arthur Conan Doyle", 989)
book2 = Book("Harry Potter", "J.K. Rowling", 1299)
book1.show_details()
book2.show_details()
```

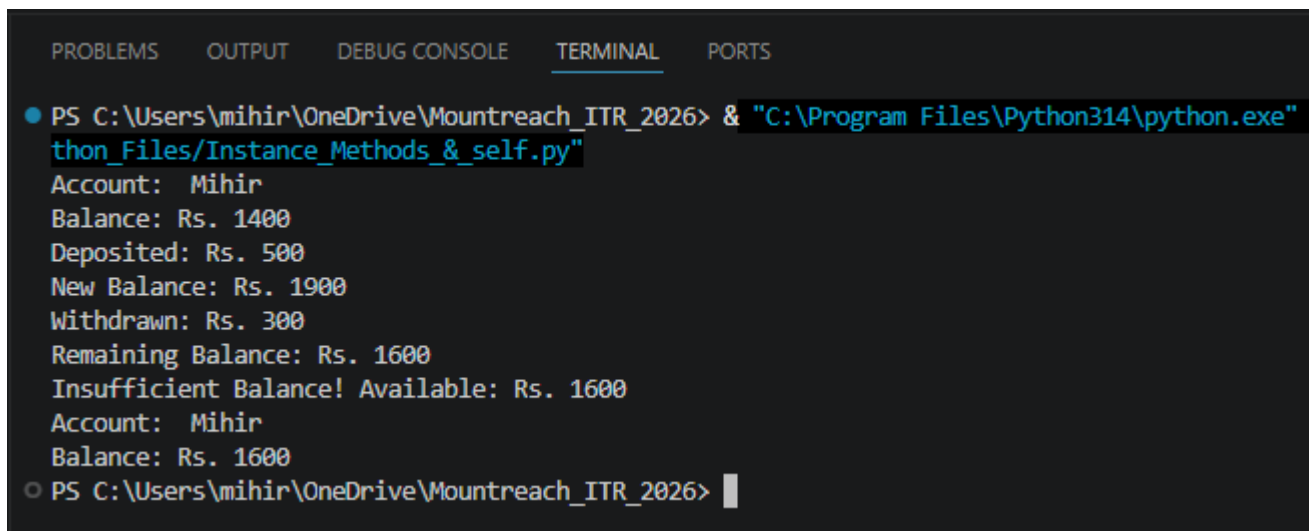
3. Instance Methods & self

Create a class BankAccount with:

- Attributes: `account_holder` and `balance` (initialized in `__init__`).
- Methods:
 - `deposit(amount)` — add amount to balance.
 - `withdraw(amount)` — subtract amount from balance (only if sufficient balance).
 - `show_balance()` — print current balance.

Create an object and demonstrate all methods.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
  thon_Files/Instance_Methods_&_self.py"
Account: Mihir
Balance: Rs. 1400
Deposited: Rs. 500
New Balance: Rs. 1900
Withdrawn: Rs. 300
Remaining Balance: Rs. 1600
Insufficient Balance! Available: Rs. 1600
Account: Mihir
Balance: Rs. 1600
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited: Rs.", amount)
        print("New Balance: Rs.", self.balance)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn: Rs.", amount)
            print("Remaining Balance: Rs.", self.balance)
        else:
```

```
        print("Insufficient Balance! Available: Rs.", self.balance)

    def show_balance(self):
        print("Account: ", self.account_holder)
        print("Balance: Rs.", self.balance)

acc = BankAccount("Mihir", 1400)
acc.show_balance()
acc.deposit(500)
acc.withdraw(300)
acc.withdraw(2000)    # Should Fail -> Insufficient Balance
acc.show_balance()
```

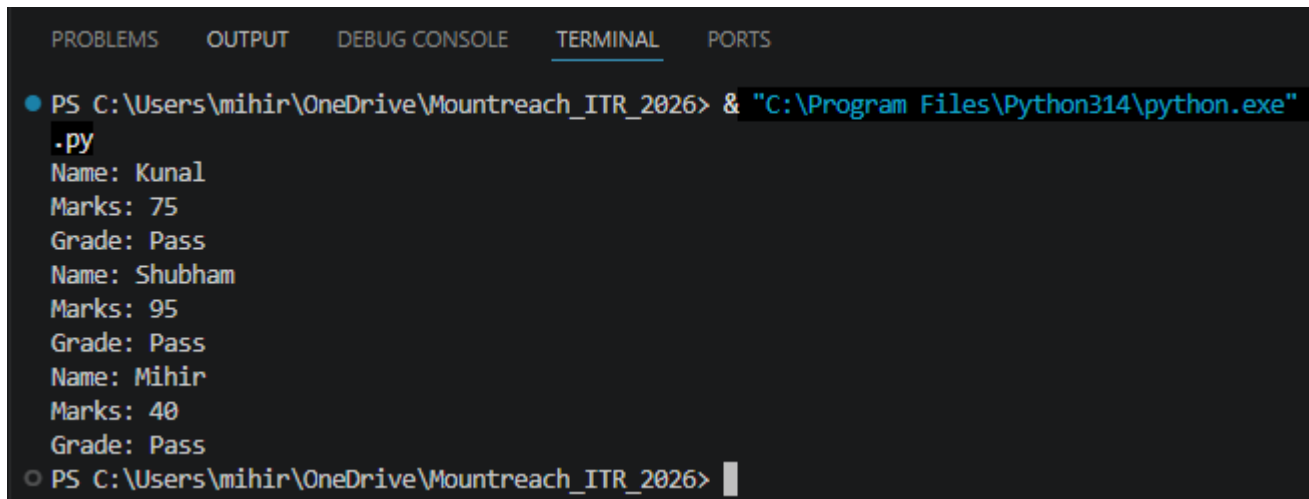
4. Method With Parameters

Create a class Student with attributes name and marks.

Add a method calculate_grade() that returns "Pass" if marks ≥ 40 , otherwise "Fail".

Create 3 student objects, display their details, and print their grades.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
.py
Name: Kunal
Marks: 75
Grade: Pass
Name: Shubham
Marks: 95
Grade: Pass
Name: Mihir
Marks: 40
Grade: Pass
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def calculate_grade(self):
        if self.marks >= 40:
            return "Pass"
        else:
            return "Fail"

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)
        print("Grade:", self.calculate_grade())

s1 = Student("Kunal", 75)
s1.display()
s2 = Student("Shubham", 95)
s2.display()
s3 = Student("Mihir", 40)
s3.display()
```

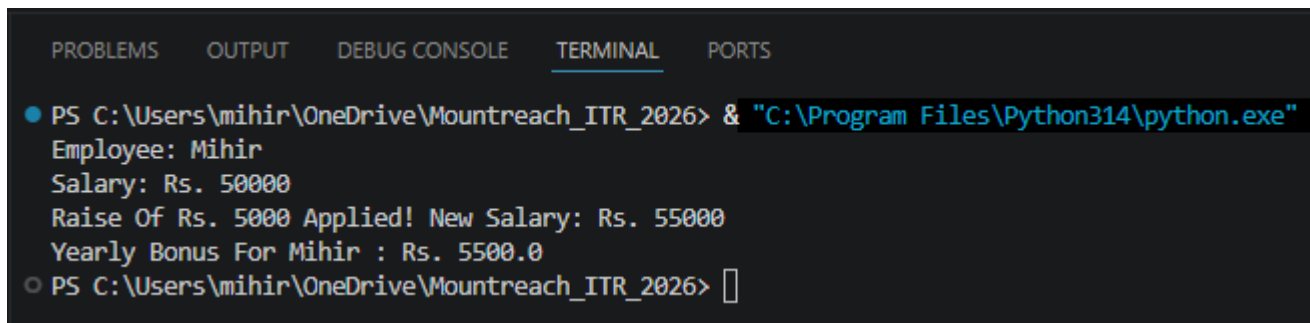
5. Multiple Methods

Create a class Employee with:

- Attributes: name, salary (use `__init__`).
- Methods:
 - `display_details()`
 - `give_raise(amount)` — increase salary and print new salary.
 - `yearly_bonus()` — return 10% of current salary.

Create an employee object and demonstrate all methods.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Employee: Mihir
Salary: Rs. 50000
Raise Of Rs. 5000 Applied! New Salary: Rs. 55000
Yearly Bonus For Mihir : Rs. 5500.0
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> █
```

Program Code:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display_details(self):
        print("Employee:", self.name)
        print("Salary: Rs.", self.salary)

    def give_raise(self, amount):
        self.salary += amount
        print("Raise Of Rs.", amount, "Applied! New Salary: Rs.", self.salary)

    def yearly_bonus(self):
        bonus = self.salary * 0.10
        print("Yearly Bonus For", self.name, ": Rs.", bonus)

emp = Employee("Mihir", 50000)
emp.display_details()
emp.give_raise(5000)
emp.yearly_bonus()
```

6. Real - World Object Modeling

Create a class `MobilePhone` with attributes: `brand`, `model`, `battery_percentage`.

Add these methods:

- `charge(percent)` — increase battery.
- `use_phone(minutes)` — reduce battery by 1% per 10 minutes.
- `show_status()` — print current status.

Create an object and simulate some usage.

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Samsung Galaxy S23 Battery: 40 %
Used For 60 Mins. Battery: 34 %
Charging... Battery Now At 64 %
Samsung Galaxy S23 Battery: 64 %
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
class MobilePhone:
    def __init__(self, brand, model, battery_percentage):
        self.brand = brand
        self.model = model
        self.battery_percentage = battery_percentage

    def charge(self, percent):
        self.battery_percentage = min(100, self.battery_percentage + percent)
        print("Charging... Battery Now At", self.battery_percentage, "%")

    def use_phone(self, minutes):
        drain = minutes // 10  # 1% Battery Drained Per 10 Minutes
        self.battery_percentage = max(0, self.battery_percentage - drain)
        print("Used For", minutes, "Mins. Battery:", self.battery_percentage, "%")

    def show_status(self):
        print(self.brand, self.model, "Battery:", self.battery_percentage, "%")

phone = MobilePhone("Samsung", "Galaxy S23", 40)
phone.show_status()
phone.use_phone(60)  # Drains 6%
phone.charge(30)     # Charges To 64%
phone.show_status()
```

7. Combining Concepts

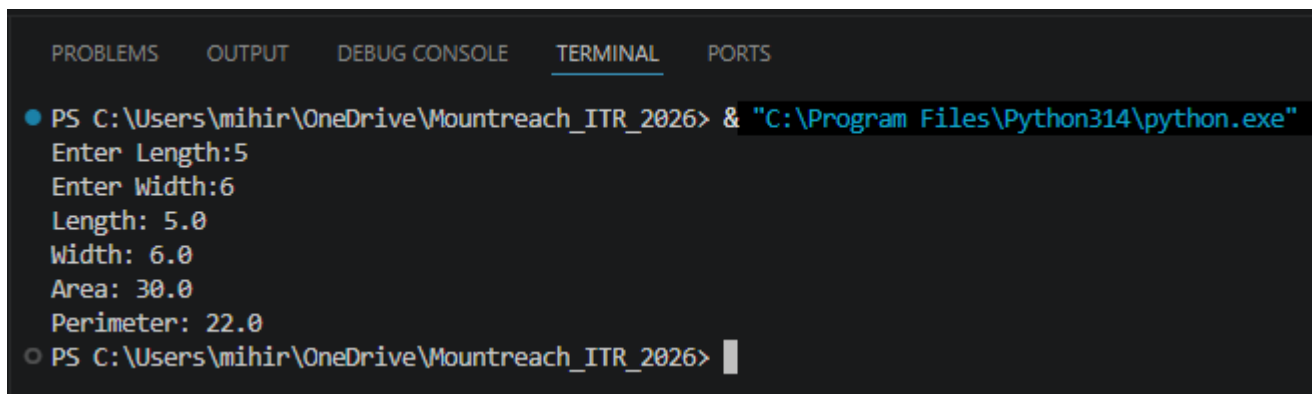
Create a class Rectangle with attributes length and width.

Add methods:

- `area()` — return the area.
- `perimeter()` — return the perimeter.
- `display()` — print length, width, area, and perimeter.

Take input from user for length and width, create object, and display results.

Output:



```
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Enter Length:5
Enter Width:6
Length: 5.0
Width: 6.0
Area: 30.0
Perimeter: 22.0
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026>
```

Program Code:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

    def display(self):
        print("Length:", self.length)
        print("Width:", self.width)
        print("Area:", self.area())
        print("Perimeter:", self.perimeter())

l = float(input("Enter Length:"))
w = float(input("Enter Width:"))
rect = Rectangle(l, w)
rect.display()
```

8. Updating Attributes

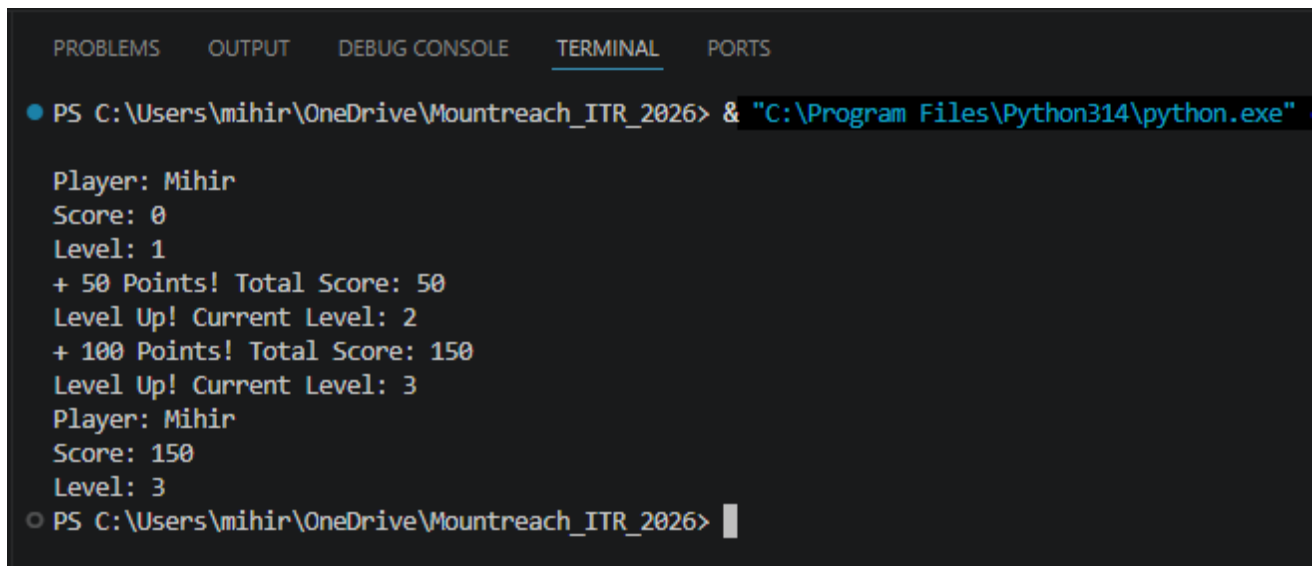
Create a class Player with attributes name, score, and level.

Add methods:

- increase_score(points)
- level up () - increase level by 1.
- show_progress()

Create an object, update score and level multiple times, and show progress.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"

Player: Mihir
Score: 0
Level: 1
+ 50 Points! Total Score: 50
Level Up! Current Level: 2
+ 100 Points! Total Score: 150
Level Up! Current Level: 3
Player: Mihir
Score: 150
Level: 3
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> |
```

Program Code:

```
class Player:
    def __init__(self, name, score=0, level=1):
        self.name = name
        self.score = score
        self.level = level

    def increase_score(self, points):
        self.score += points
        print("+", points, "Points! Total Score:", self.score)

    def level_up(self):
        self.level += 1
        print("Level Up! Current Level:", self.level)

    def show_progress(self):
        print("Player:", self.name)
```

```
        print("Score:", self.score)
        print("Level:", self.level)

player = Player("Mihir")
player.show_progress()
player.increase_score(50)
player.level_up()
player.increase_score(100)
player.level_up()
player.show_progress()
```

9. Debugging OOP Code

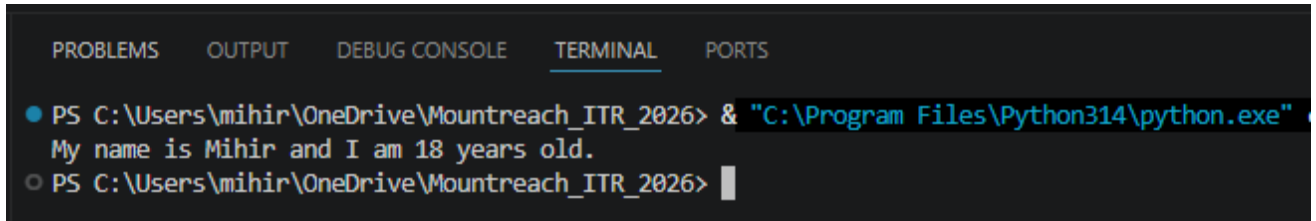
The following code has errors. Correct it and add proper comments explaining the fixes:

```
class Person:
    def __init__(name, age):
        name = name
        age = age

    def introduce():
        print("My name is" name "and I am" age "years old.")

p1 = Person("Rahul", 25)
p1.introduce()
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
My name is Mihir and I am 18 years old.
○ PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> █
```

Program Code:

```
class Person:
    def __init__(self, name, age):    # Fix 1: Added 'self'
        self.name = name            # Fix 2: self.name Stores The Attribute
        self.age = age              # Fix 2: elf.age Stores The Attribute

    def introduce(self):             # Fix 3: Added 'self'
        # Fix 4: f-string For Proper String Formatting
        print(f"My name is {self.name} and I am {self.age} years old.")

p1 = Person("Mihir", 18)
p1.introduce()
```

10. Mini Project - Combined Concepts

Create a **Simple Library Management System** using OOP:

Design a class Book with:

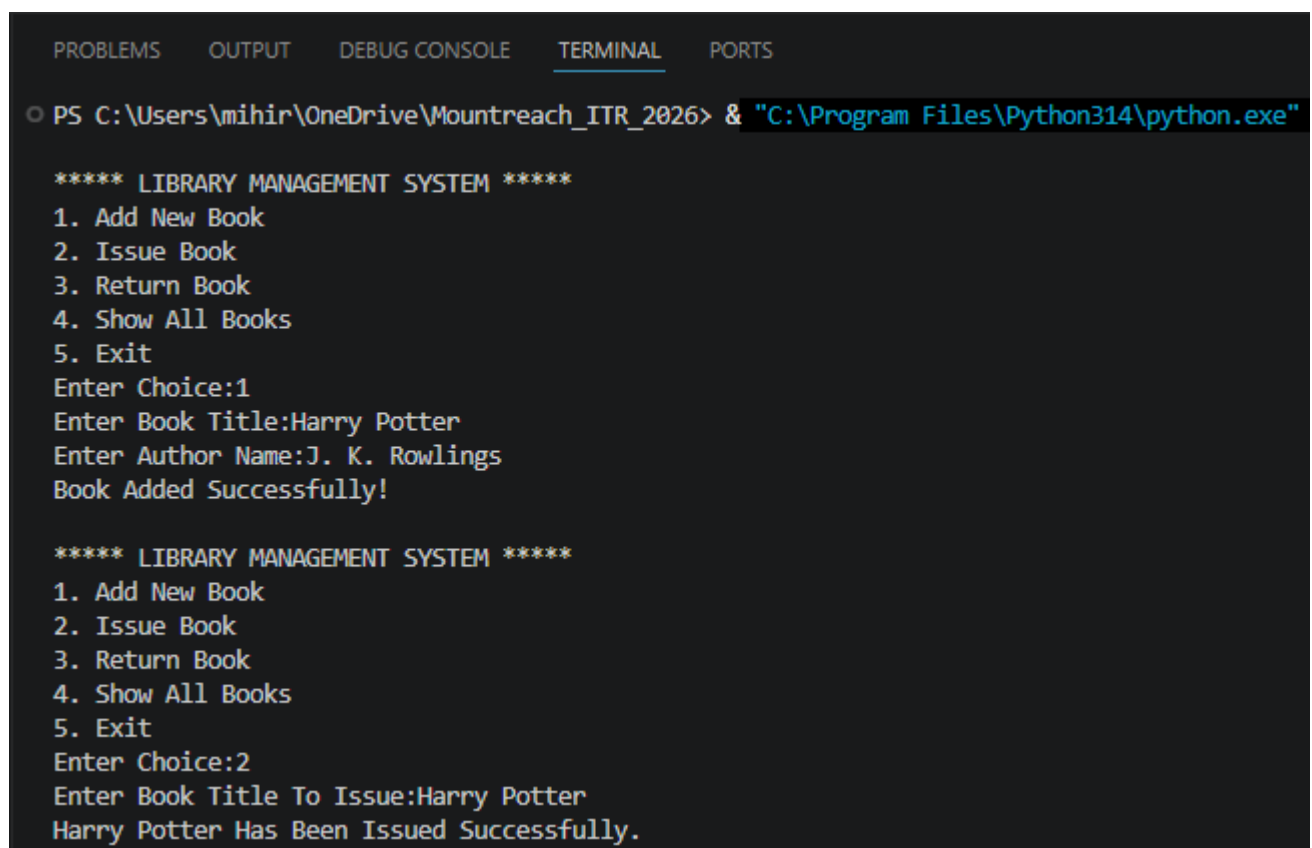
- Attributes: title, author, status ("Available" or "Issued").
- Methods: issue_book(), return_book(), show_info().

Create multiple book objects and demonstrate issuing/returning books with proper messages.

Use a menu (while loop) to allow the user to:

- a) Add a new book
- b) Issue a book
- c) Return a book
- d) Show all books
- e) Exit

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\mihir\OneDrive\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"

***** LIBRARY MANAGEMENT SYSTEM *****
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice:1
Enter Book Title:Harry Potter
Enter Author Name:J. K. Rowlings
Book Added Successfully!

***** LIBRARY MANAGEMENT SYSTEM *****
1. Add New Book
2. Issue Book
3. Return Book
4. Show All Books
5. Exit
Enter Choice:2
Enter Book Title To Issue:Harry Potter
Harry Potter Has Been Issued Successfully.
```

Program Code:

```
class Book:
    def __init__(self):
        self.title = input("Enter Book Title:")
        self.author = input("Enter Author Name:")
        self.status = "Available"

    def issue_book(self):
        if self.status == "Available":
            self.status = "Issued"
            print(self.title, "Has Been Issued Successfully.")
        else:
            print(self.title, "Is Already Issued.")

    def return_book(self):
        if self.status == "Issued":
            self.status = "Available"
            print(self.title, "Has Been Returned Successfully.")
        else:
            print(self.title, "Was Not Issued.")

    def show_info(self):
        print("\nTitle :", self.title)
        print("Author :", self.author)
        print("Status :", self.status)

library = []

def add_book():
    book = Book()
    library.append(book)
    print("Book Added Successfully!")

def issue_book():
    title = input("Enter Book Title To Issue:")

    for book in library:
        if book.title == title:
            book.issue_book()
            return

    print("Book Not Found!")

def return_book():
    title = input("Enter Book Title To Return:")

    for book in library:
        if book.title == title:
            book.return_book()
```

```

        return

    print("Book Not Found!")

def show_all_books():
    if len(library) == 0:
        print("No Books Available.")
    else:
        for book in library:
            book.show_info()

while True:
    print("\n***** LIBRARY MANAGEMENT SYSTEM *****")
    print("1. Add New Book")
    print("2. Issue Book")
    print("3. Return Book")
    print("4. Show All Books")
    print("5. Exit")

    choice = input("Enter Choice:")

    if choice == "1":
        add_book()

    elif choice == "2":
        issue_book()

    elif choice == "3":
        return_book()

    elif choice == "4":
        show_all_books()

    elif choice == "5":
        print("Exiting Program...")
        break

    else:
        print("Invalid Choice!")

```